Nutritional Info Identifier: A Mobile Food Detection and Nutrition Lookup System

Members: Ada Taing, Kimberly Eak, Trina Das
Course: COMP-4990
Instructor: Dr. Jessica Chen

Date: April 6, 2026

Table of Contents

1. Introduction and Motivation

The Nutritional Info Identifier is a mobile application that uses image recognition technology to detect food items captured through the device's camera. Once a food item is identified, the application retrieves nutritional information such as calories, protein, carbohydrates, fat, and provides helpful tips related to the detected food.

Our motivation for this project was to address the growing need for simple and accessible tools that support healthy eating habits through real-time dietary tracking. Many existing nutrition tracking methods require users to manually search for foods or enter calorie values, which can be time-consuming, inconvenient, and discouraging for consistent use. As a result, many individuals struggle to maintain accurate records of their daily nutritional intake.

Therefore, this application provides users with a fast and intuitive way to understand the nutritional content of their meals, and ultimately help reduce the barriers associated with traditional food tracking methods.

2. Background Study

Our research focused on Convolutional Neural Networks (CNNs) and dietary applications. We examined existing solutions such as MyFitnessPal and observed that many current tools rely heavily on manual data entry or barcode scanning, which can reduce convenience and efficiency for users. We also explored the feasibility of using pre-trained deep learning models, such as MobileNet and Inception, for food image classification tasks.

Although modern computer vision models perform well in general object recognition, accurately distinguishing between visually similar food items (for example, differentiating an apple from a peach) remains a significant challenge. Variations in lighting conditions, camera angles, and background environments can further affect model performance. These challenges highlight the importance of selecting an appropriate model architecture and dataset when developing an image-based food recognition system.

3. Methodology / Model / Approach

Our approach utilizes a supervised learning model for image-based food recognition. The system is designed to automatically identify food items from images captured through a mobile interface and retrieve corresponding nutritional information.

3.1 Input:

JPEG or PNG images captured using the mobile application camera.

3.2 Processing

The captured images are converted into an appropriate format and passed to a trained YOLO (You Only Look Once) object detection model. The model processes the image in a single pass and outputs the predicted food label along with a confidence score. The images used during training were resized and normalized to improve model performance and ensure consistent input dimensions.

3.3 Data Mapping

Once a food item is identified, the detected label is sent to the backend server, which queries the USDA FoodData Central API to retrieve nutritional values based on a standardized 100g serving size. The system then formats the nutritional information and sends it back to the mobile application for display.

3.4 Refinement

Initially, the goal was to recognize a wide range of food categories. However, due to dataset limitations and project time constraints, the scope was refined to focus primarily on fruit items. Narrowing the dataset improved model accuracy and confidence scores, resulting in more reliable predictions within the available timeframe.

4. Design Document

4.1 System Architecture

The Nutritional Info Identifier system is designed using a front-end/back-end architecture. The front end is responsible for the user experience, including image capture and result display, while the backend handles image analysis, model inference, nutrition retrieval, and data caching. The repository structure clearly separates these two major parts into different application folders, showing a modular design.

When a user captures an image, the mobile application sends the image to the backend server through an API request. The backend processes the image using the YOLO food detection model to determine the predicted food label. Once the food item is identified, the system checks a local SQLite cache to determine whether nutritional data has already been retrieved previously. If the data exists in the cache, it is returned immediately to improve efficiency. Otherwise, the backend queries the USDA FoodData Central API to retrieve the nutritional information, stores the result in the cache for future requests, and sends the formatted response back to the mobile application.

Separating the system into frontend and backend components allows the application to remain modular, making it easier to maintain, update, and expand in the future.

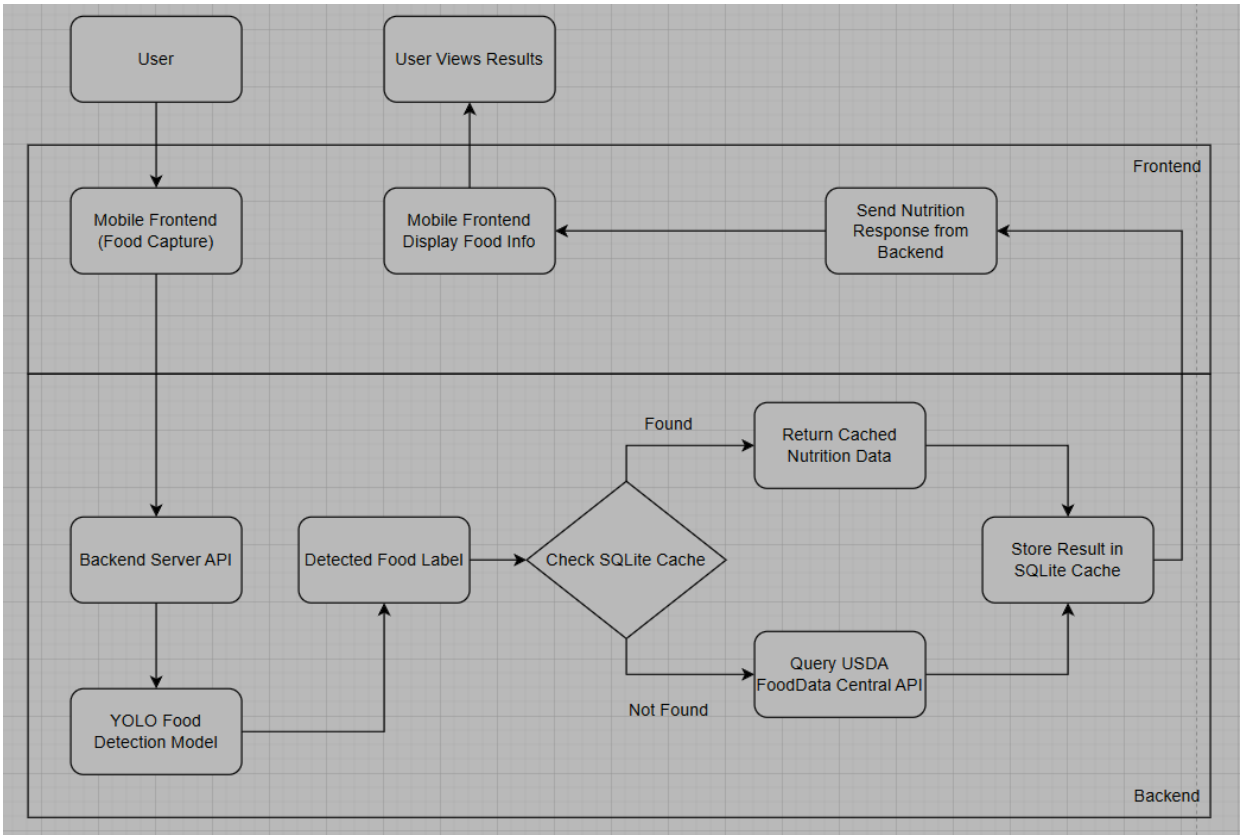


Diagram 1: High Level System Architecture Diagram

4.2 Pipeline Design

The internal pipeline of the application can be described as follows:

1. The user opens the mobile application.
2. The user captures an image of a food item using the device camera.
3. The captured image is uploaded to the backend through the FastAPI /analyze endpoint.
4. The backend receives and decodes the image file into a format suitable for processing.
5. The YOLO food detection model analyzes the image and predicts the food label along with a confidence score.
6. The detected food label is extracted from the model output.
7. The system checks the SQLite nutrition cache to determine whether nutritional data for the detected food already exists.
8. If the data exists in the cache, the stored nutritional values are retrieved.
9. If the data is not found in the cache, the backend queries the USDA FoodData Central API to retrieve nutritional information based on a standardized 100g serving size.
10. The retrieved nutrition data is stored in the SQLite cache to improve performance for future requests.
11. The structured nutrition response is sent back to the mobile application.

12. The mobile interface displays the detected food label along with its nutritional information for the user.

This pipeline is consistent with the backend components shown in server.py, which imports the model, image processing tools, API request support, and SQLite functions in one integrated service.

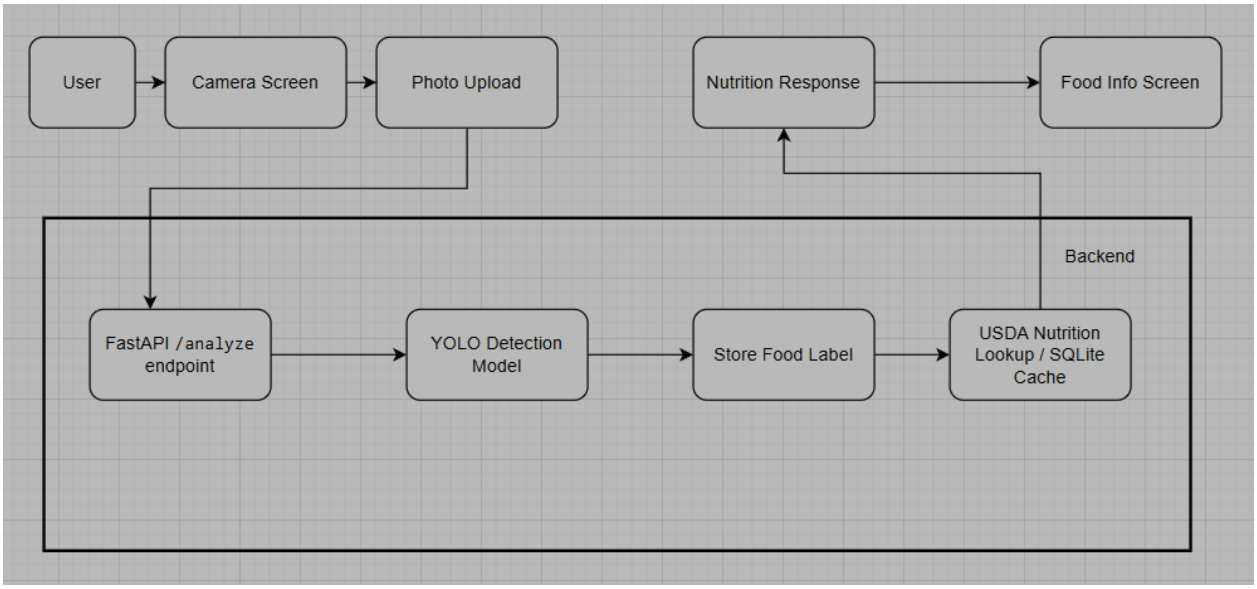


Diagram 2: Data Pipeline

4.3 Database Organization

The project uses SQLite as a lightweight local database to store cached nutrition results. Instead of implementing a full database management system, SQLite was selected due to its simplicity, efficiency, and seamless integration within the backend environment.

The database stores the detected food label, the corresponding nutritional information, and a timestamp indicating when the data was last retrieved or updated. This caching mechanism allows the application to reuse previously obtained nutrition data without repeatedly sending requests to the USDA FoodData Central API.

By reducing the number of external API calls, the system improves response time, decreases network dependency, and enhances overall application performance. Additionally, local caching helps ensure more consistent results even if external services are temporarily unavailable.

5. Implementation Details

Front End (Mobile Application - React Native with Expo):

- Developed as a cross-platform mobile application using React Native and Expo
- Provides a user-friendly interface that allows users to capture food images using the device camera
- Sends captured images to the backend server through an HTTP request
- Displays detected food labels, nutritional values, and dietary tips dynamically
- Handles user interaction, navigation between screens, and formatting of returned nutrition data
- Includes error handling to manage cases where no food is detected or the server request fails

Back End (FastAPI - Python Server):

- Implemented using FastAPI to create a REST API that processes image data
- Receives images from the mobile application and converts them into a format suitable for model input
- Performs image preprocessing such as resizing and normalization before inference
- Connects all system components, acting as the central logic layer of the application
- Sends structured JSON responses containing detected food labels and nutritional information back to the front end

AI Model (YOLO Object Detection Model):

- Utilizes a YOLO-based supervised learning model for food detection
- The model analyzes the input image and identifies the most probable food item present
- Returns the detected food label along with a confidence score
- The project scope was refined to focus primarily on fruit categories to improve prediction accuracy within the project timeframe
- Model inference is performed on the backend server to ensure efficient processing

6. Datasets used

For the food detection component of the project, two primary image sources were used: manually collected real-life photos and the Open Images V7 dataset.

Real-life photos were captured manually to reflect realistic usage conditions of the mobile application. These images include variations in lighting, background environments, camera angles, and image quality, helping the model better generalize to real-world scenarios. Using real images ensures that the model is trained on inputs similar to what users would capture through the mobile device camera.

The Open Images V7 dataset was used to increase the overall size and diversity of the training data. This dataset provides labeled images across a wide range of object categories, including fruits relevant to the project scope. Incorporating Open Images V7 allowed the model to learn from a larger number of examples, improving its ability to recognize visual patterns and distinguish between similar-looking food items.

By combining manually captured images with a structured public dataset, the project benefits from both realistic application-specific data and broader image diversity. This balanced approach helps improve model robustness and overall detection performance while remaining feasible within the project timeframe.

7. API Document

The project uses backend APIs to connect the mobile application with the food detection model and nutrition information source. APIs allow the front end and back end to communicate by sending requests and receiving structured responses.

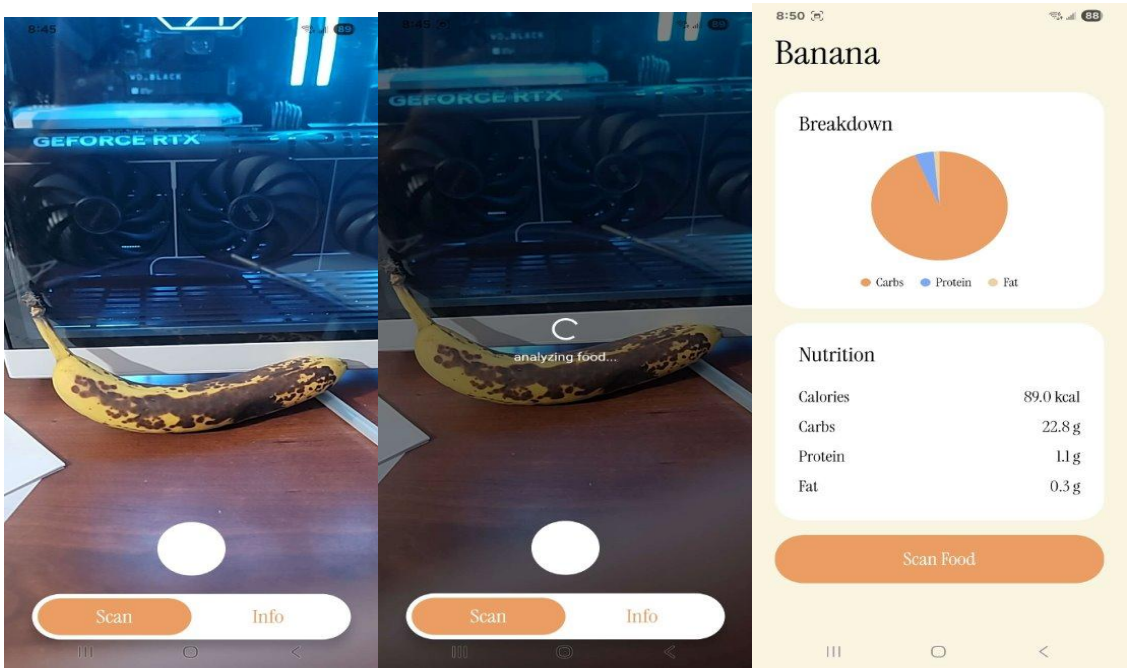
The main API developed in this project is a FastAPI endpoint that receives an image from the mobile application. When the user takes a photo, the image is sent to the backend using an HTTP POST request. The backend API processes the image and runs the YOLO detection model to identify the food item present.

Once the food label is detected, the backend uses the USDA FoodData Central API to retrieve nutrition information related to that food. The USDA API provides standardized nutrition values such as calories, protein, carbohydrates, and fat based on a typical 100g serving size.

The backend then combines the detected food label and nutrition information into a structured JSON response. This response is sent back to the mobile application, where the results are displayed to the user in a clear and readable format.

APIs used in the project:

- FastAPI (custom backend API)
- USDA FoodData Central API



Display 3: Application UI showing the overall usage

8. Source Code

The project source code is organized into two main sections: the mobile application (front end) and the server (back end). The structure separates user interface functionality from data processing logic, making the code easier to navigate and modify.

The front-end code includes components related to camera functionality, user interface layout, and displaying the returned results. The back-end code contains the implementation of the detection pipeline, communication with external services, and local data handling.

Key code contributions include:

- Camera interface and image submission logic in the mobile application
- Backend endpoint for receiving and processing image requests
- Integration of the YOLO model for food identification
- Implementation of structured JSON responses for consistent communication between components
- SQLite caching logic to store previously retrieved nutrition results

All code was stored and updated using Github:
https://github.com/AdaTaing/Nutritional_Info_Identifier

9. Discussion

9.1 Challenges

One of the main challenges faced was model accuracy. Training a model capable of recognizing many different food types requires a large and diverse dataset, which can be difficult to obtain within a limited timeframe. To address this, the project scope was refined to focus primarily on fruit detection. This allowed the model to achieve higher confidence scores while remaining feasible within the project timeline. Additionally, we had to filter through many of the food items we had to make sure that the quality of the images were good enough for the model to be trained on.

Another challenge was connecting the mobile application to the backend server. Since the mobile app was tested on a physical device while the backend was hosted locally on a computer, network configuration issues occurred when attempting to send image requests. The application initially attempted to communicate through localhost, which refers to the device itself rather than the computer running the server. This required configuring the correct local IP address so the mobile application could successfully communicate with the backend through FastAPI and Uvicorn.

Another difficulty involved dynamically updating the user interface when new images were captured. Initially, previously detected results were not resetting properly when a new photo was taken, causing outdated nutrition information to remain displayed. This required adjusting state management in the front-end application to ensure results update correctly each time a new image is processed.

9.2 What we Learned

Through the challenges encountered during development, we learned the importance of properly planning project scope and accounting for time constraints when working with machine learning and full-stack systems. Training an accurate image recognition model requires a large, high-quality dataset, which can be time-consuming to collect and preprocess. As a result, we adjusted our original plan of detecting many food types and instead focused primarily on fruit detection. This allowed us to produce more reliable results within the available project timeline. This experience showed us the importance of setting realistic goals early and being willing to refine the project scope when necessary.

10. Conclusion and Future Work

The Nutritional Info Identifier project demonstrates how machine learning and mobile technology can be combined to create a practical tool for real-time nutrition tracking. The system successfully integrates image capture, YOLO-based food detection, and nutrition retrieval into a working prototype that provides users with quick access to dietary information. Due to project

time constraints, the scope was refined to focus on a smaller set of food categories in order to improve model reliability and ensure successful system integration.

Future improvements could include expanding the dataset to support more food types, improving model accuracy, and adding features such as portion size estimation or extended nutrition tracking. The project provides a strong foundation for further development of accessible, automated dietary tools.

11. References

Github Repository: https://github.com/AdaTaing/Nutritional_Info_Identifier

Datasets:

- Google Open Images v7: https://storage.googleapis.com/openimages/web/index.html
- USDA: https://fdc.nal.usda.gov/

Tools:

- FiftyOne Zoo: https://docs.voxel51.com/dataset_zoo/index.html
- YOLO: https://docs.ultralytics.com/models/yolov8/
- Python: https://www.python.org/
- Google Colab: https://developers.google.com/colab
- Visual Studio Code: https://code.visualstudio.com/
- React Native/Expo: https://reactnative.dev/
- RoboFlow: https://roboflow.com/
- FastAPI: https://fastapi.tiangolo.com/
- Uvicorn: https://uvicorn.dev/